

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <sys/types.h>
6 #include <sys/ipc.h>
7 #include <sys/msg.h>
8
9 struct message {
10     long msg_type;
11     char msg_text[100];
12 };
13
14 int main() {
15     key_t key = ftok("msgqfile", 65); // Generate a unique key for the message queue
16
17     // Create a new message queue (or get the identifier of an existing one)
18     int msgid = msgget(key, IPC_CREAT | 0666);
19     if (msgid == -1) {
20         perror("msgget");
21         exit(EXIT_FAILURE);
22     }
23
24     struct message msg;
25     msg.msg_type = 1; // Message type (can be any positive number)
26
27     // Producer: Send a message to the message queue
28     strcpy(msg.msg_text, "Hello, message queue!");
29     if (msgsnd(msgid, (void*)&msg, sizeof(msg.msg_text), IPC_NOWAIT) == -1) {
30         perror("msgsnd");
31         exit(EXIT_FAILURE);
32     }
33     printf("Producer: Data sent to message queue: %s\n", msg.msg_text);
34
35     // Consumer: Receive a message from the message queue
36     if (msgrcv(msgid, (void*)&msg, sizeof(msg.msg_text), 1, 0) == -1) {
37         perror("msgrcv");
38         exit(EXIT_FAILURE);
39     }
40     printf("Consumer: Data received from message queue: %s\n", msg.msg_text);
41
42     // Remove the message queue
```

Producer: Data sent to message queue: Hello, message queue!

Consumer: Data received from message queue: Hello, message queue!

=== Code Execution Successful ===

```
43     if (msgctl(msgid, IPC_RMID, NULL) == -1) {
44         perror("msgctl");
45         exit(EXIT_FAILURE);
46     }
47
48     return 0;
49 }
```